

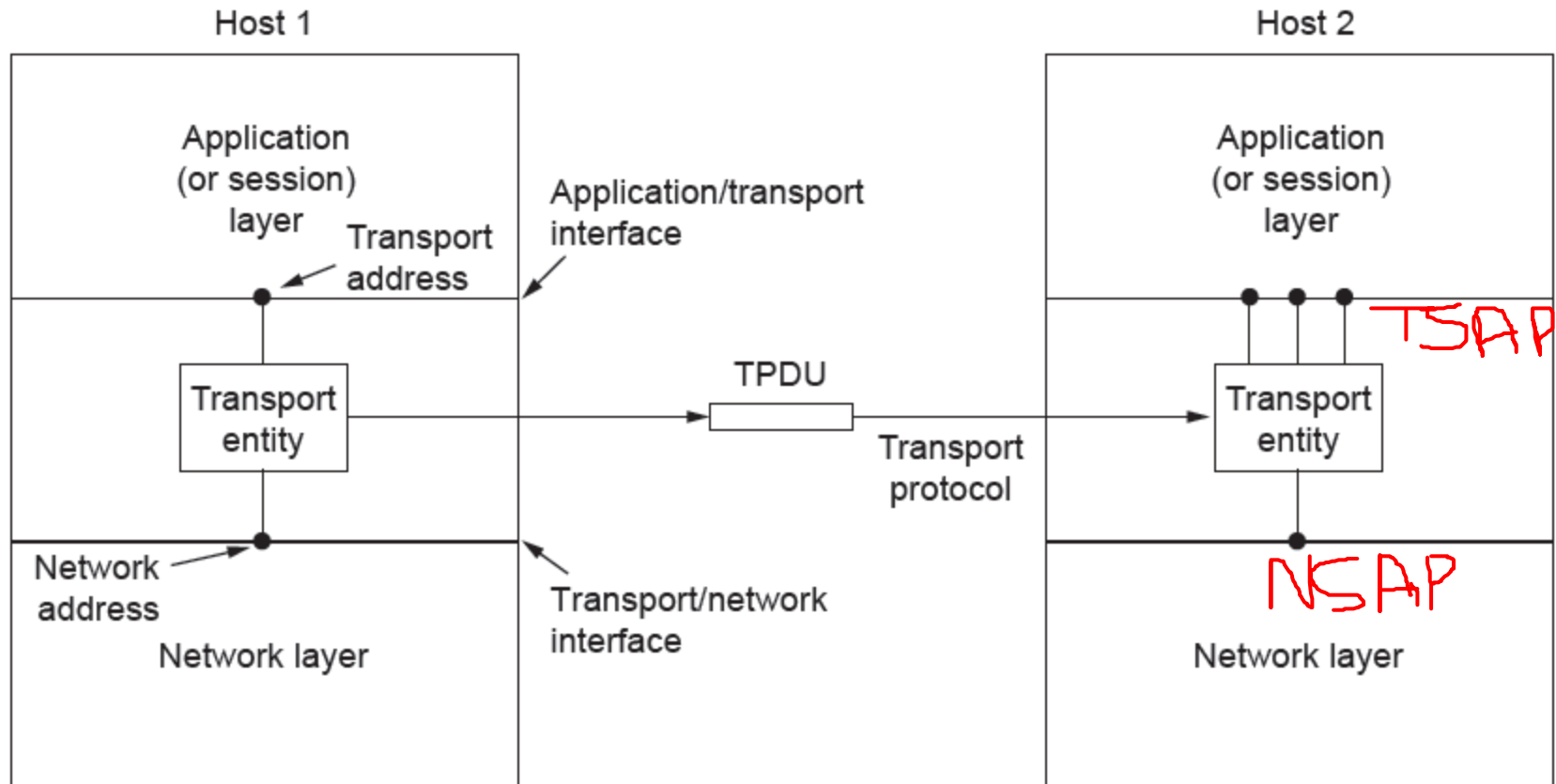
The Transport Layer

Chapter 6

Transport Service

- Upper Layer Services
- Transport Service Primitives
- Berkeley Sockets
- Example of Socket Programming:
Internet File Server

Services Provided to the Upper Layers



The network, transport, and application layers

Port no is a 16 bit field

Services Provided to the Upper Layers

- Qualitative distinction between layers 1 through 4 and layer(s) above 4
- The bottom four layers can be seen as the **transport service provider**, whereas the upper layer(s) are the **transport service user**
- It forms the major boundary between the provider and user of the reliable data transmission service

Transport Service Primitives (1)

- To allow users to access the transport service, the transport layer must provide some operations to application programs, that is, a transport service interface
- Each transport service has its own interface
- The connection-oriented transport service, in contrast, is reliable. Of course, real networks are not error-free, but that is precisely the purpose of the transport layer—to provide a reliable service on top of an unreliable network
- As an example, consider two processes on a single machine connected by a pipe in UNIX (or any other interprocess communication facility). They assume the connection between them is 100% perfect. They do not want to know about acknowledgements, lost packets, congestion, or anything at all like that.
- The connection-oriented transport service is all about—hiding the imperfections of the network service so that user processes can just assume the existence of an error-free bit stream even when they are on different machines.
- The transport layer can also provide unreliable (datagram) service

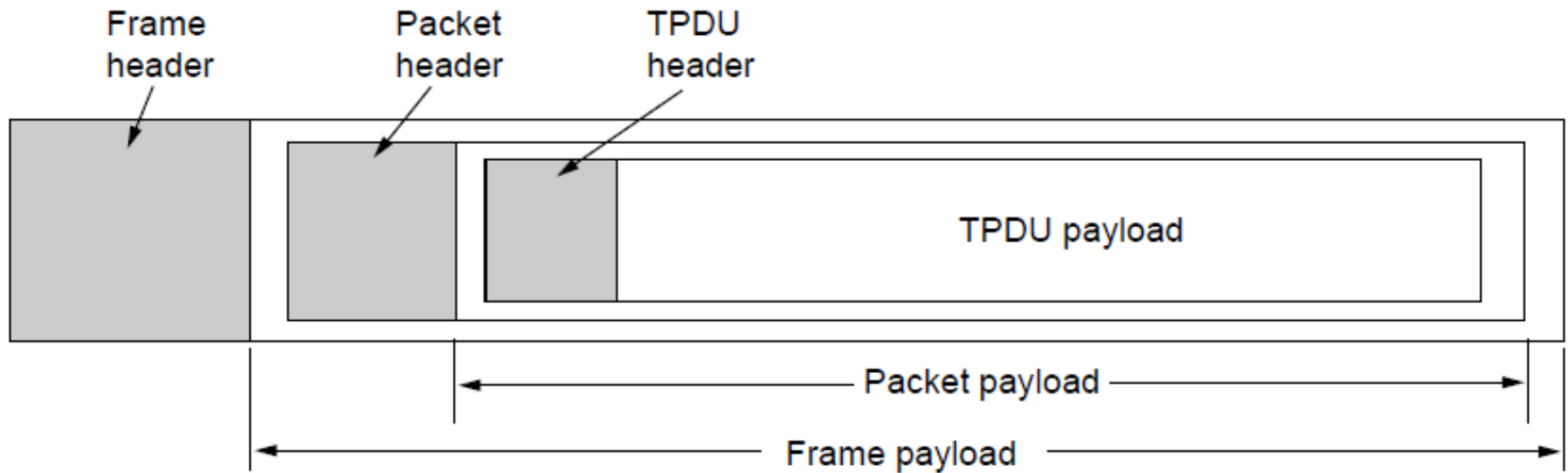
Transport Service Primitives (2)

- A second difference between the network service and transport service is whom the services are intended for
- Very few users write their own transport entities, and thus few users or programs ever see the bare network service. In contrast, many programs (and thus programmers writing socket applications) see the transport primitives

Primitive	Packet sent	Meaning
LISTEN	(none)	Block until some process tries to connect
CONNECT	CONNECTION REQ.	Actively attempt to establish a connection
SEND	DATA	Send information
RECEIVE	(none)	Block until a DATA packet arrives
DISCONNECT	DISCONNECTION REQ.	This side wants to release the connection

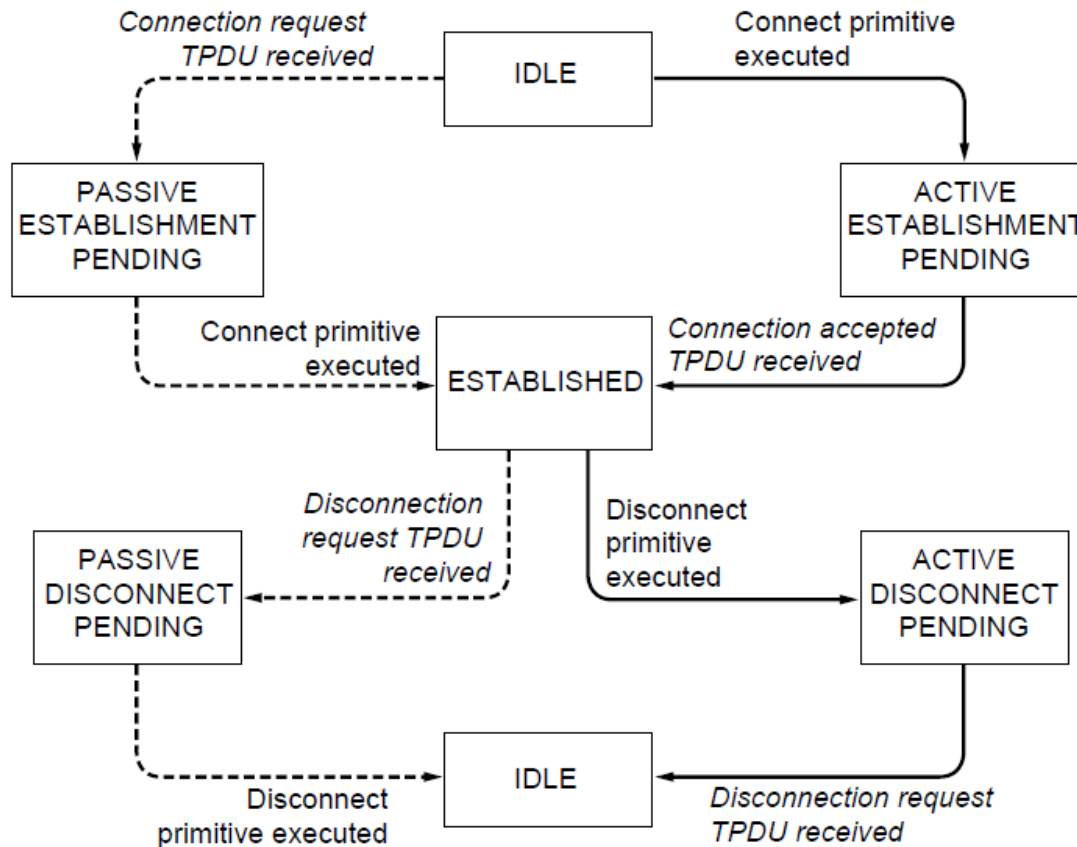
The primitives for a simple transport service

Transport Service Primitives (3)



Nesting of TPDUs, packets, and frames.

Connection Establishment



A state diagram for a simple connection management scheme. Transitions labeled in italics are caused by packet arrivals. The solid lines show the client's state sequence. The dashed lines show the server's state sequence.

Berkeley Sockets (2)

Primitive	Meaning
SOCKET	Create a new communication end point
BIND	Associate a local address with a socket
LISTEN	Announce willingness to accept connections; give queue size
ACCEPT	Passively establish an incoming connection
CONNECT	Actively attempt to establish a connection
SEND	Send some data over the connection
RECEIVE	Receive some data from the connection
CLOSE	Release the connection

The socket primitives for TCP

Transport Service Primitives (1)

- The first four primitives in the list are executed in that order by servers.
- The SOCKET primitive creates a new endpoint and allocates table space for it within the transport entity. The parameters of the call specify the addressing format to be used, the type of service desired (e.g., reliable byte stream), and the protocol
- Newly created sockets are assigned network addresses with BIND primitive

Example of Socket Programming: An Internet File Server (1)

```
/* This page contains a client program that can request a file from the server program
 * on the next page. The server responds by sending the whole file.
 */

#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>

#define SERVER_PORT 12345          /* arbitrary, but client & server must agree */
#define BUF_SIZE 4096             /* block transfer size */

int main(int argc, char **argv)
{
    int c, s, bytes;
    char buf[BUF_SIZE];            /* buffer for incoming file */
    struct hostent *h;             /* info about server */
    struct sockaddr_in channel;    /* holds IP address */

    . . .
```

Client code using sockets

Example of Socket Programming: An Internet File Server (2)

• • •

```
if (argc != 3) fatal("Usage: client server-name file-name");
h = gethostbyname(argv[1]);           /* look up host's IP address */
if (!h) fatal("gethostbyname failed");

s = socket(PF_INET, SOCK_STREAM, IPPROTO_TCP);
if (s < 0) fatal("socket");
memset(&channel, 0, sizeof(channel));
channel.sin_family= AF_INET;
memcpy(&channel.sin_addr.s_addr, h->h_addr, h->h_length);
channel.sin_port= htons(SERVER_PORT);

c = connect(s, (struct sockaddr *) &channel, sizeof(channel));
if (c < 0) fatal("connect failed");
```

• • •

Client code using sockets

Example of Socket Programming: An Internet File Server (3)

...

```
c = connect(s, (struct sockaddr *) &channel, sizeof(channel));
if (c < 0) fatal("connect failed");

/* Connection is now established. Send file name including 0 byte at end. */
write(s, argv[2], strlen(argv[2])+1);

/* Go get the file and write it to standard output. */
while (1) {
    bytes = read(s, buf, BUF_SIZE);      /* read from socket */
    if (bytes <= 0) exit(0);              /* check for end of file */
    write(1, buf, bytes);                 /* write to standard output */
}
}

fatal(char *string)
{
    printf("%s\n", string);
    exit(1);
}
```

Client code using sockets

Example of Socket Programming: An Internet File Server (4)

```
#include <sys/types.h>
#include <sys/fcntl.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>

#define SERVER_PORT 12345
#define BUF_SIZE 4096
#define QUEUE_SIZE 10

int main(int argc, char *argv[])
{
    int s, b, l, fd, sa, bytes, on = 1;
    char buf[BUF_SIZE];
    struct sockaddr_in channel;

    . . .
```

/* This is the server code */

/* arbitrary, but client & server must agree */
/* block transfer size */

/* buffer for outgoing file */
/* holds IP address */

Server code

Example of Socket Programming: An Internet File Server (5)

• • •

```
/* Build address structure to bind to socket. */
memset(&channel, 0, sizeof(channel));      /* zero channel */
channel.sin_family = AF_INET;
channel.sin_addr.s_addr = htonl(INADDR_ANY);
channel.sin_port = htons(SERVER_PORT);

/* Passive open. Wait for connection. */
s = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP); /* create socket */
if (s < 0) fatal("socket failed");
setsockopt(s, SOL_SOCKET, SO_REUSEADDR, (char *) &on, sizeof(on));

b = bind(s, (struct sockaddr *) &channel, sizeof(channel));
if (b < 0) fatal("bind failed");

l = listen(s, QUEUE_SIZE);                /* specify queue size */
if (l < 0) fatal("listen failed");
```

• • •

Server code

Example of Socket Programming: An Internet File Server (6)

...

```
/* Socket is now set up and bound. Wait for connection and process it. */
while (1) {
    sa = accept(s, 0, 0);                /* block for connection request */
    if (sa < 0) fatal("accept failed");

    read(sa, buf, BUF_SIZE);            /* read file name from socket */

    /* Get and return the file. */
    fd = open(buf, O_RDONLY);            /* open the file to be sent back */
    if (fd < 0) fatal("open failed");

    while (1) {
        bytes = read(fd, buf, BUF_SIZE); /* read from file */
        if (bytes <= 0) break;           /* check for end of file */
        write(sa, buf, bytes);           /* write bytes to socket */
    }
    close(fd);                          /* close file */
    close(sa);                          /* close connection */
}
}
```

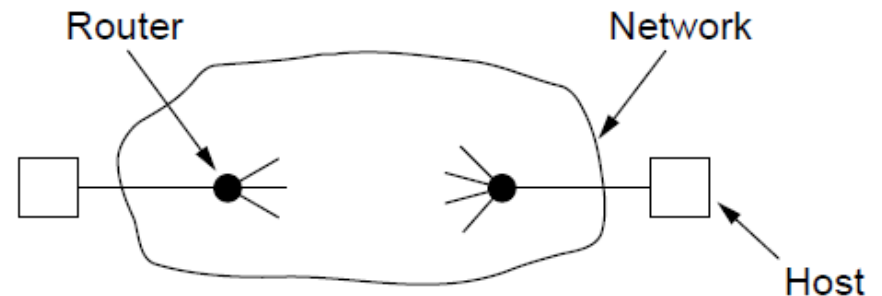
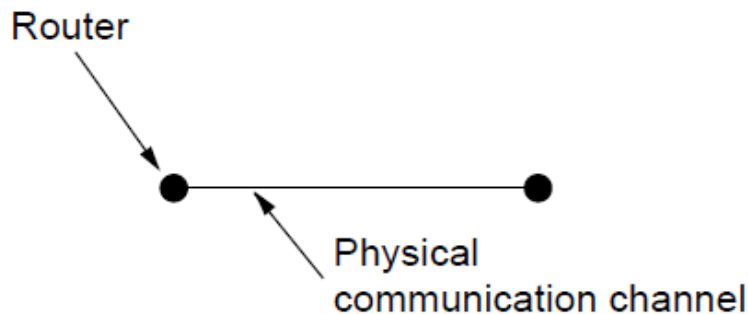
Server code

Elements of Transport Protocols (1)

- Addressing
- Connection establishment
- Connection release
- Error control and flow control
- Multiplexing
- Crash recovery

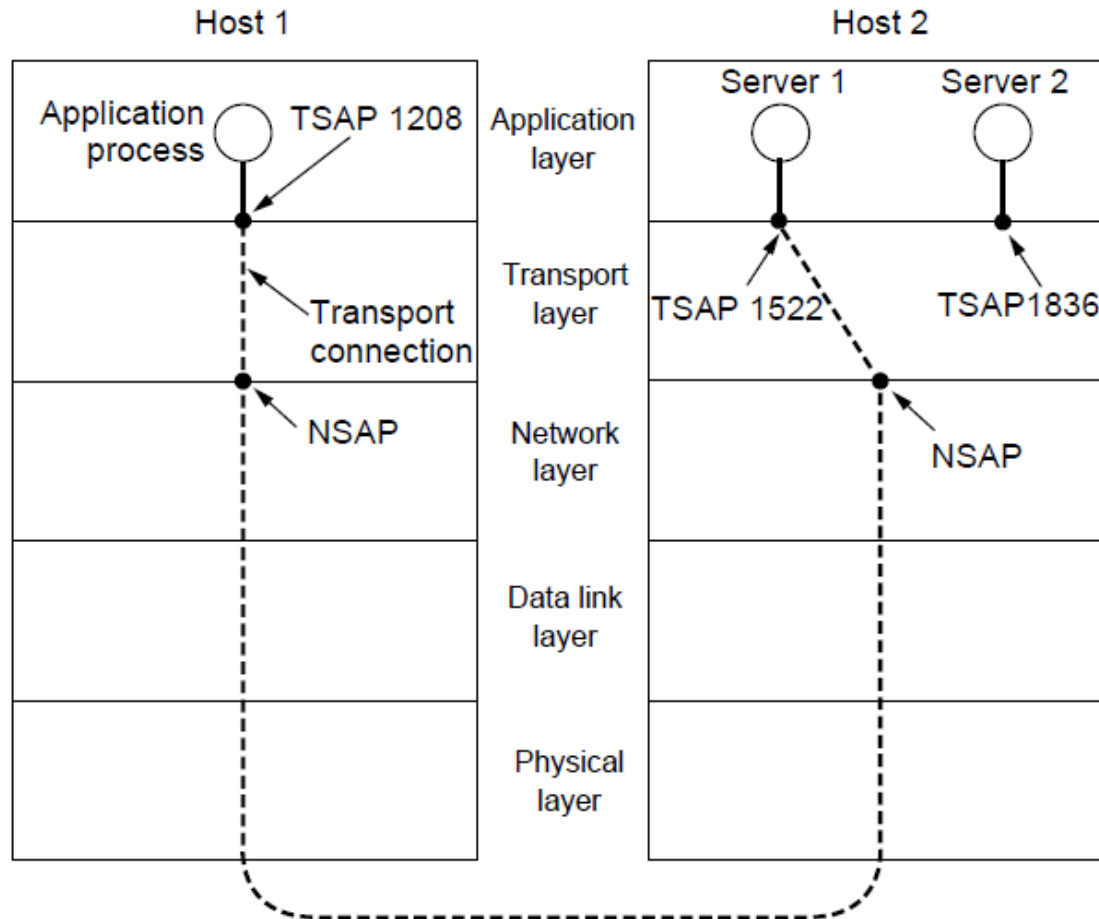
Elements of Transport Protocols (2)

- The process of establishing a connection over the wire is simple
- The potential existence of storage capacity in the network
- Buffering and flow control are needed in both layers, but the presence of a large and varying number of connections with fluctuating bandwidth as the connections compete with each other may require a different approach than we used in the data link layer.



a) Environment of the data link layer. (a)
(b) Environment of the transport layer. (b)

Addressing (1)



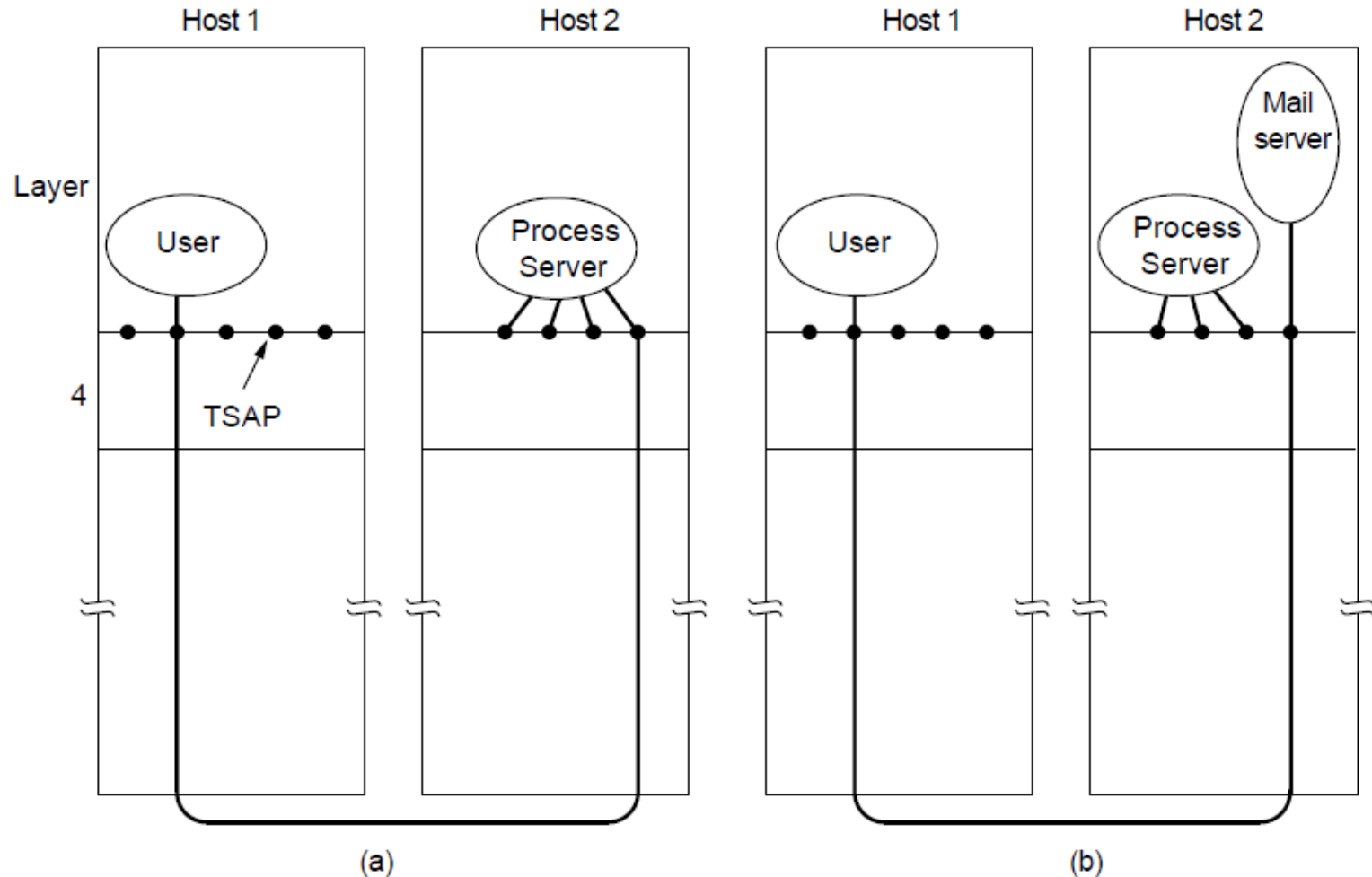
TSAPs, NSAPs, and transport connections

How does the host 1 know that the mail server is attached to TSAP 1522?

Addressing (2)

- How does the user process on host 1 know that the mail server is attached to TSAP 1522?
- One possibility is to fix that service on that port
- Use **Portmapper** - The user sends a message specifying the service name, and the portmapper sends back the TSAP address
- Use **ProcessServer** - acts as a proxy for less heavily used servers. It listens to a set of ports at the same time, then spawn the requested server

Addressing (3)



How a user process in host 1 establishes a connection with a mail server in host 2 via a process server.

Connection Establishment (1)

- If network is fully congested, neither data or ack are delivered, leading to multiple retransmissions.
- If user crashes and start using the same set of sequence used earlier, it might lead to duplicate packet delivery problem
- Fund transfer example
- Solution can be –
 1. discard the port numbers used before crash
 2. Assign each connection unique number

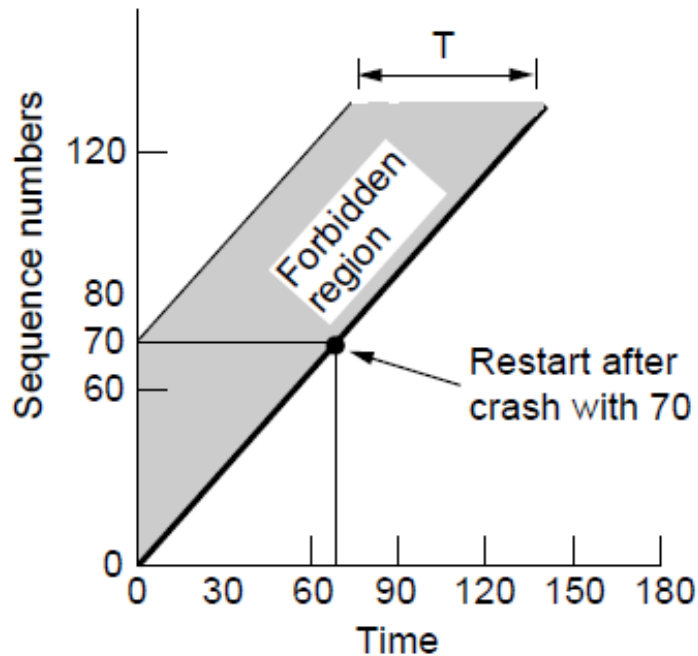
Connection Establishment (1)

Techniques for restricting packet lifetime

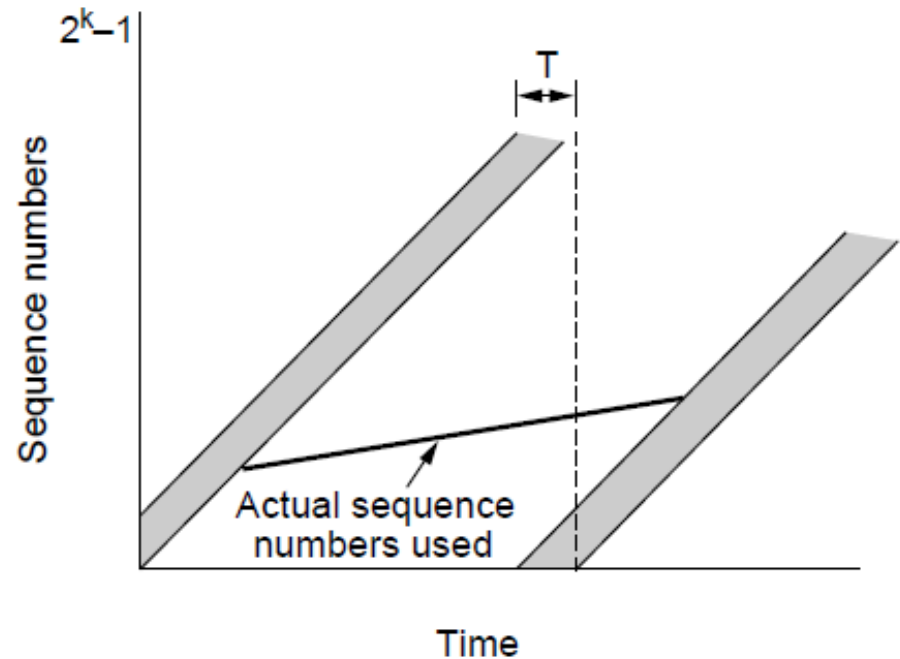
- Restricted network design.
- Putting a hop counter in each packet.
- Timestamping each packet.

Connection Establishment (2)

- * Forbidden region shows the times for which segment seq numbers are illegal
- * If any segment is sent with a sequence number in this region, it could be delayed and impersonate a different packet with the same sequence number that will be issued slightly later



(a)



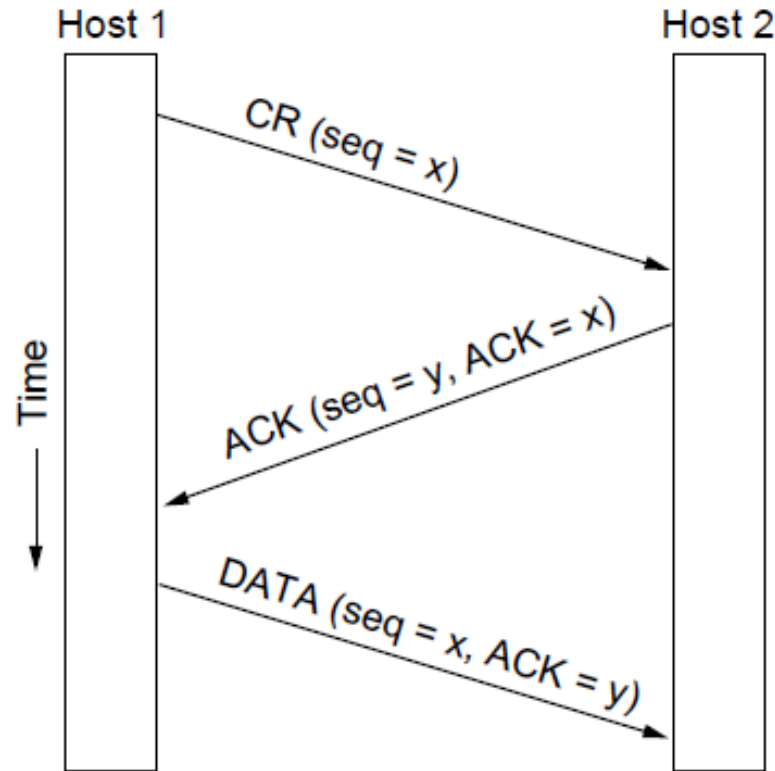
(b)

- TPDUs may not enter the forbidden region.
- The resynchronization problem.

Connection Establishment

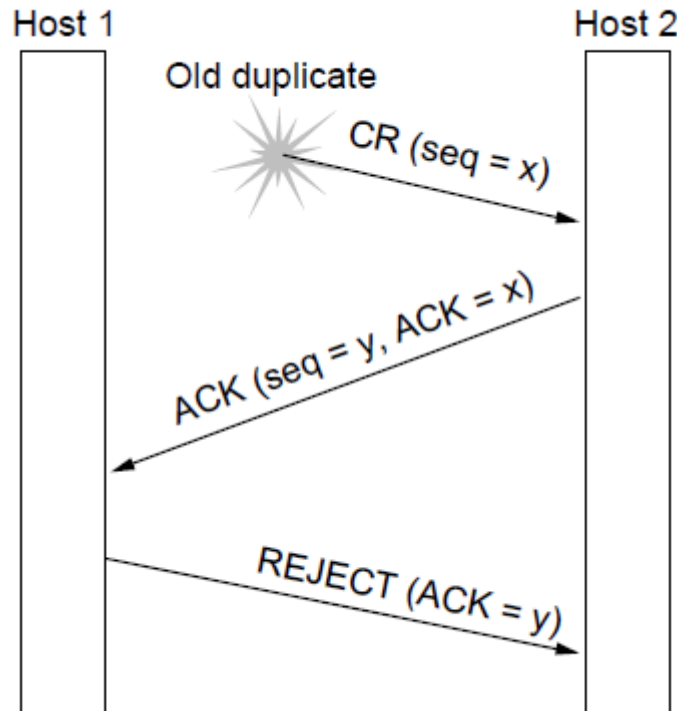
- Each connection starts numbering its segments with a different initial sequence number.
- The sequence number should be so large that by the time the sequence numbers wrap around, old segments with the same sequence numbers are long gone.
- Linear relationship of sequence numbers with time is shown in part (a) of the figure in previous slide.
- If a host sends too much data too fast on a newly opened connection, the actual sequence number versus time curve may rise more steeply than the initial sequence number versus time curve, causing the sequence number to enter the forbidden region.
- Any data rate less than the clock rate, the curve of actual sequence number versus time will eventually run into forbidden region.

Connection Establishment (3)



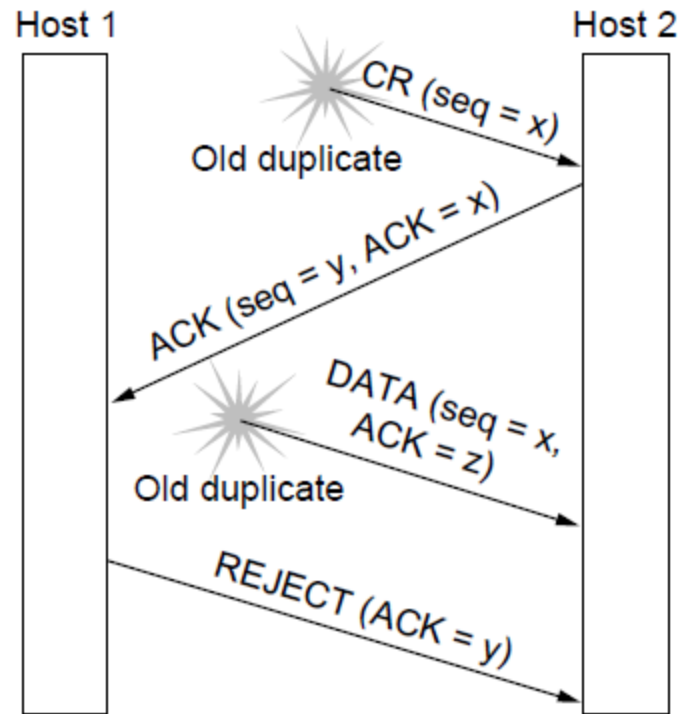
Three protocol scenarios for establishing a connection using a three-way handshake. CR denotes CONNECTION REQUEST. Normal operation.

Connection Establishment (4)



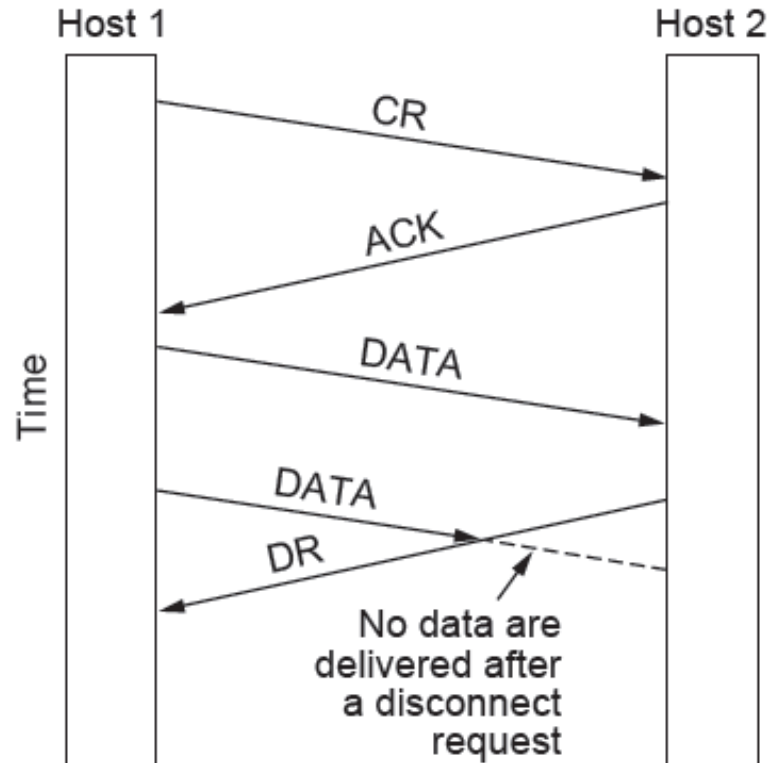
Three protocol scenarios for establishing a connection using a three-way handshake. CR denotes CONNECTION REQUEST. Old duplicate CONNECTION REQUEST appearing out of nowhere.

Connection Establishment (5)



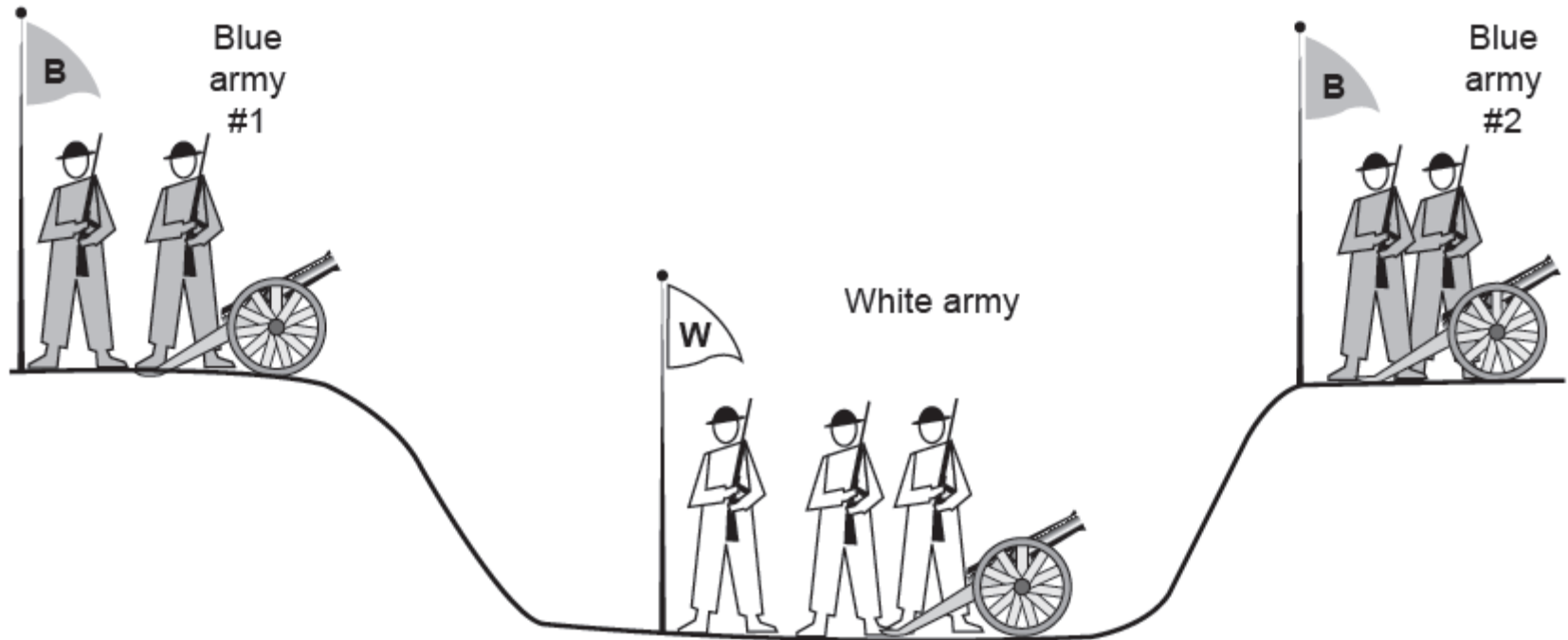
Three protocol scenarios for establishing a connection using a three-way handshake. CR denotes CONNECTION REQUEST. Duplicate CONNECTION REQUEST and duplicate ACK

Connection Release (1)



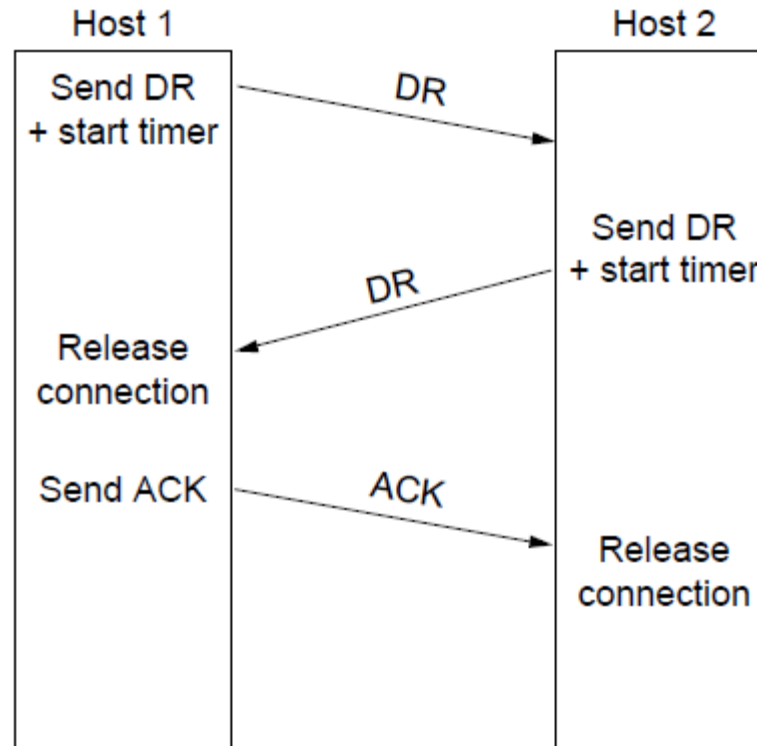
Abrupt disconnection with loss of data

Connection Release (2)



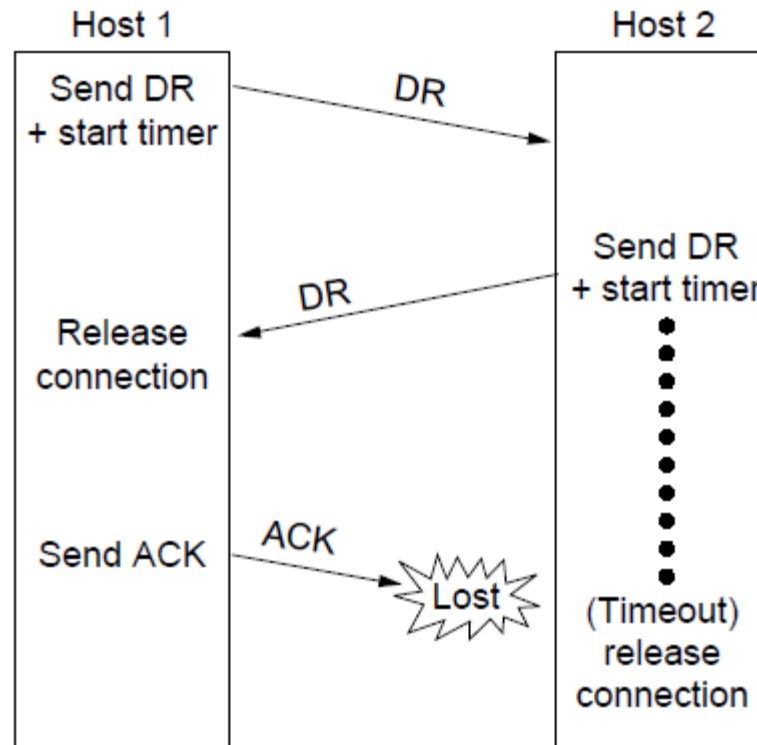
The two-army problem

Connection Release (3)



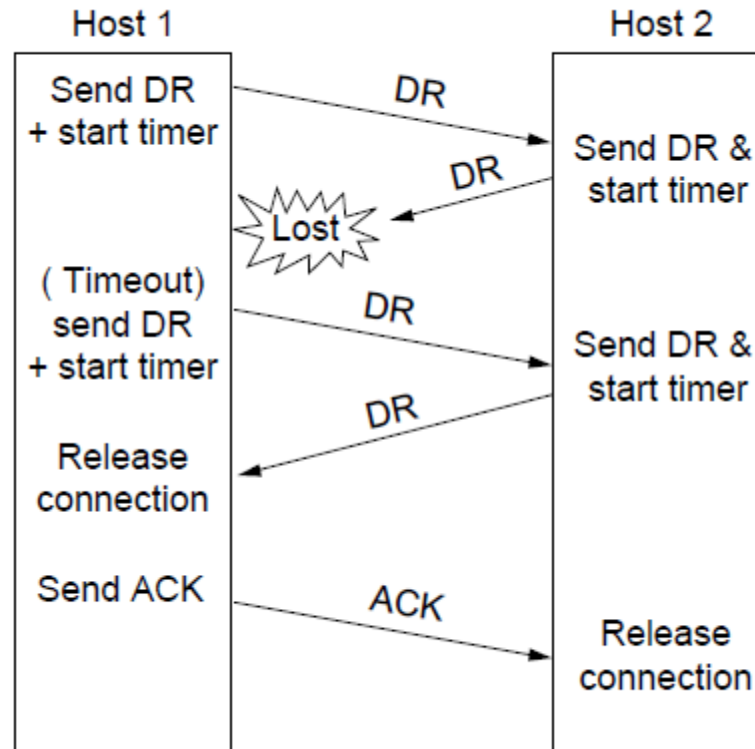
Four protocol scenarios for releasing a connection.
(a) Normal case of three-way handshake

Connection Release (4)



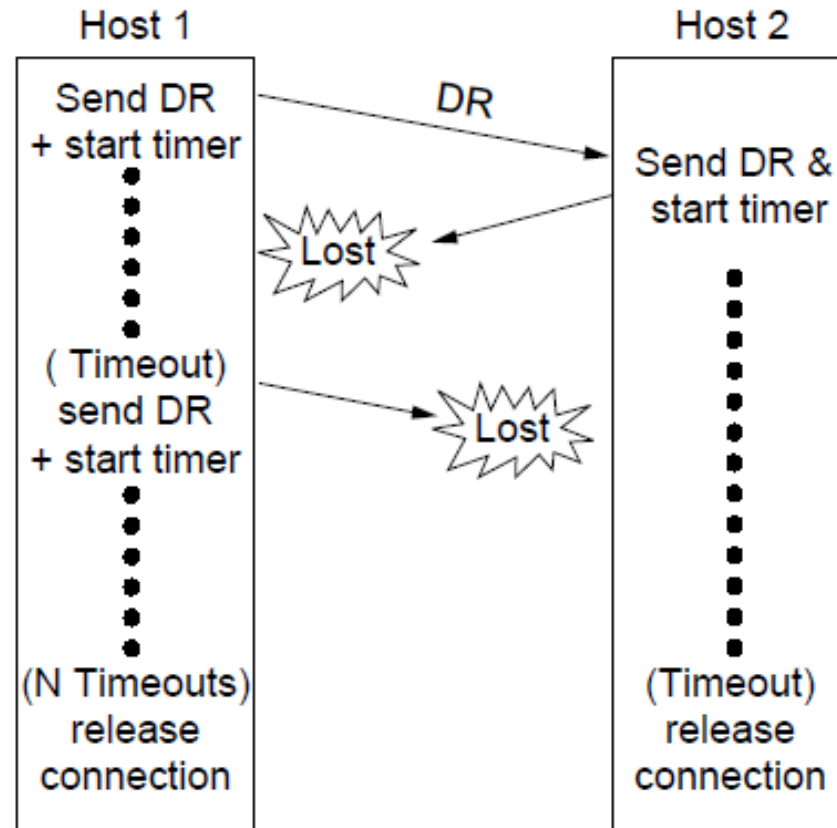
Four protocol scenarios for releasing a connection.
(b) Final ACK lost.

Connection Release (5)



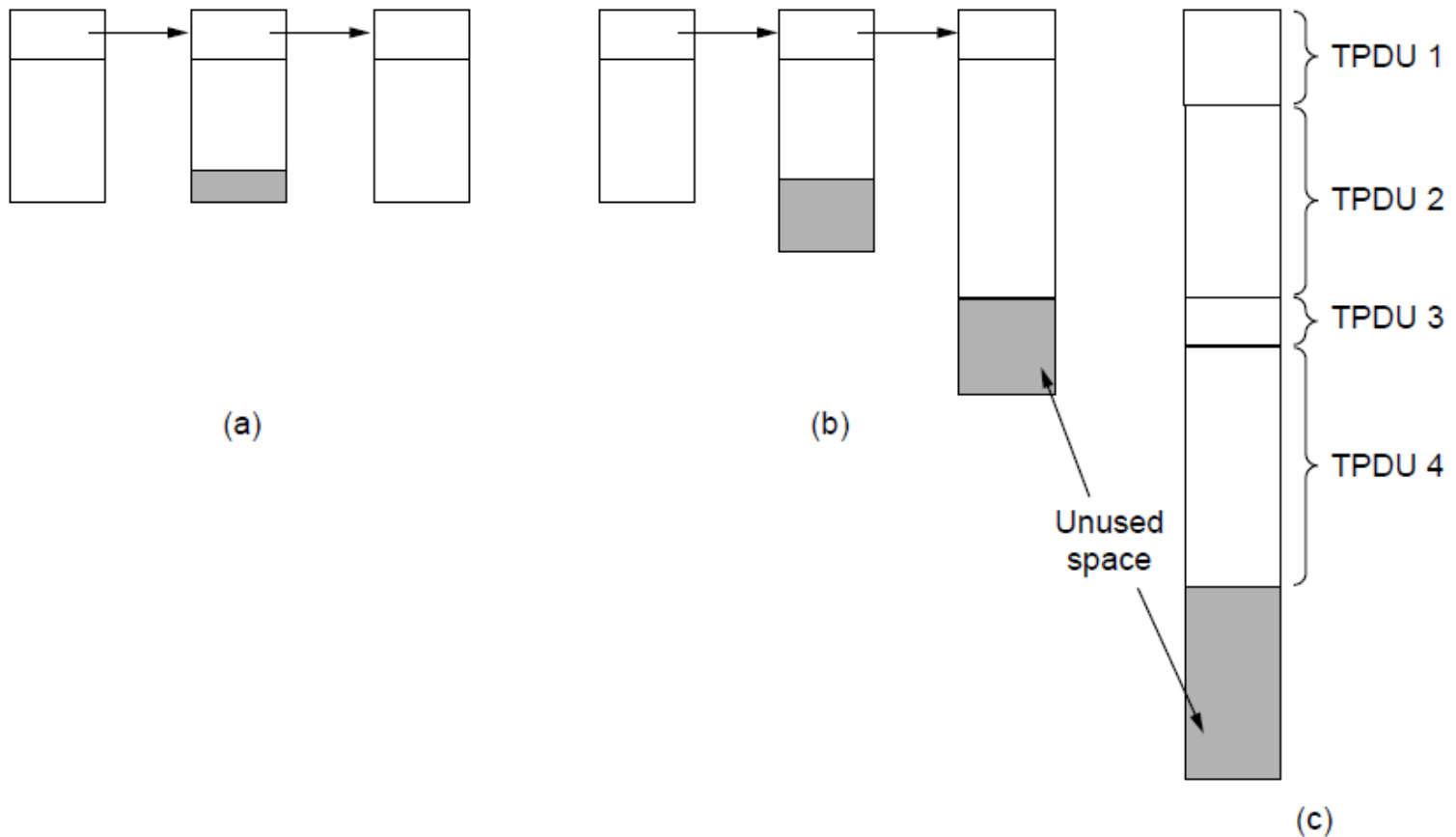
Four protocol scenarios for releasing a connection.
(c) Response lost

Connection Release (6)



Four protocol scenarios for releasing a connection.
(d) Response lost and subsequent DRs lost.

Error Control and Flow Control (1)



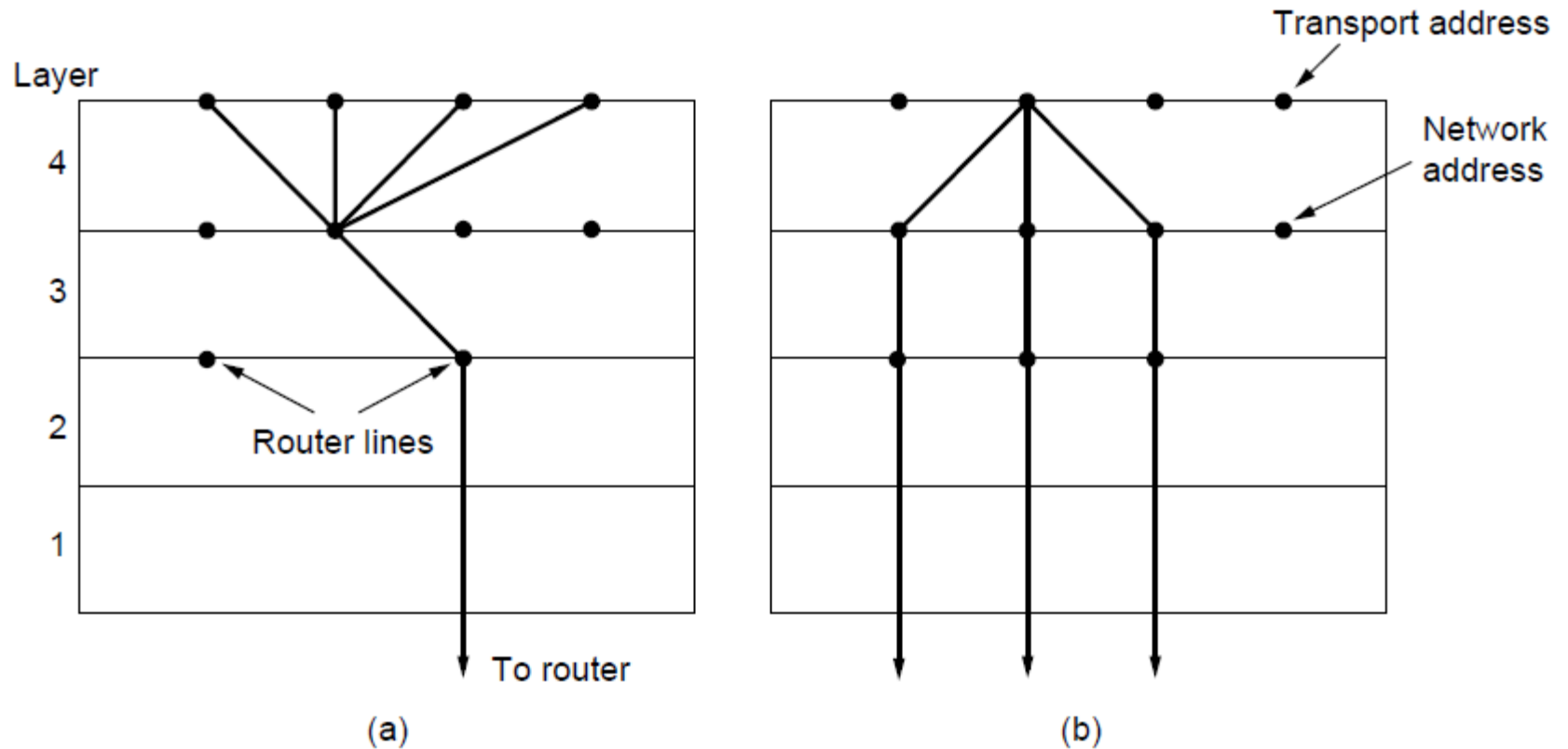
- a) (a) Chained fixed-size buffers. (b) Chained variable-sized buffers. (c) One large circular buffer per connection.

Error Control and Flow Control (2)

	<u>A</u>	<u>Message</u>	<u>B</u>	<u>Comments</u>
1	→	< request 8 buffers>	→	A wants 8 buffers
2	←	<ack = 15, buf = 4>	←	B grants messages 0-3 only
3	→	<seq = 0, data = m0>	→	A has 3 buffers left now
4	→	<seq = 1, data = m1>	→	A has 2 buffers left now
5	→	<seq = 2, data = m2>	...	Message lost but A thinks it has 1 left
6	←	<ack = 1, buf = 3>	←	B acknowledges 0 and 1, permits 2-4
7	→	<seq = 3, data = m3>	→	A has 1 buffer left
8	→	<seq = 4, data = m4>	→	A has 0 buffers left, and must stop
9	→	<seq = 2, data = m2>	→	A times out and retransmits
10	←	<ack = 4, buf = 0>	←	Everything acknowledged, but A still blocked
11	←	<ack = 4, buf = 1>	←	A may now send 5
12	←	<ack = 4, buf = 2>	←	B found a new buffer somewhere
13	→	<seq = 5, data = m5>	→	A has 1 buffer left
14	→	<seq = 6, data = m6>	→	A is now blocked again
15	←	<ack = 6, buf = 0>	←	A is still blocked
16	...	<ack = 6, buf = 4>	←	Potential deadlock

Dynamic buffer allocation. The arrows show the direction of transmission. An ellipsis (...) indicates a lost TPDU

Multiplexing



(a) Multiplexing. (b) Inverse multiplexing.

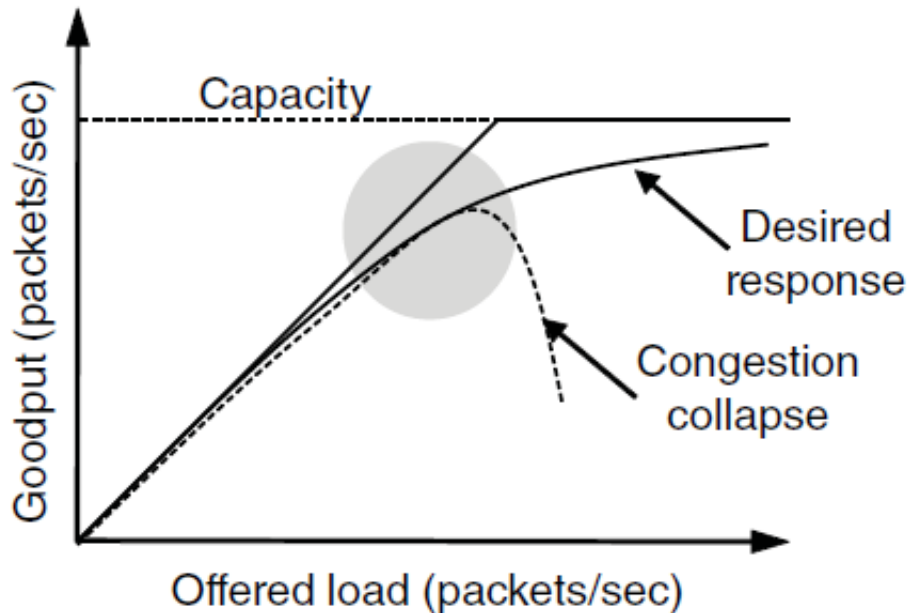
Crash Recovery

- In case of network or router crash, protocols can easily recover by retransmitting the lost segment.
- In case of host crash, it is desirable that client should continue when server crash and quickly reboot.
- In an attempt to recover its previous status, the server might send a broadcast segment to all other hosts, announcing that it has just crashed and requesting that its client to inform the status of all open connections.
- Each client can be in one of two states: one segment outstanding S1 or no segment outstanding S0.
- Server can implement either ACK then write or write then ACK approach (Write means deliver data to upper layer)

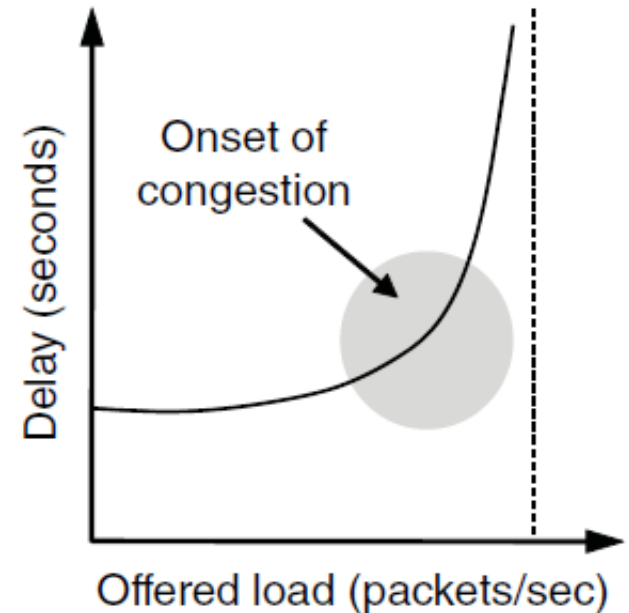
Congestion Control

- Desirable bandwidth allocation
- Regulating the sending rate

Desirable Bandwidth Allocation (1)



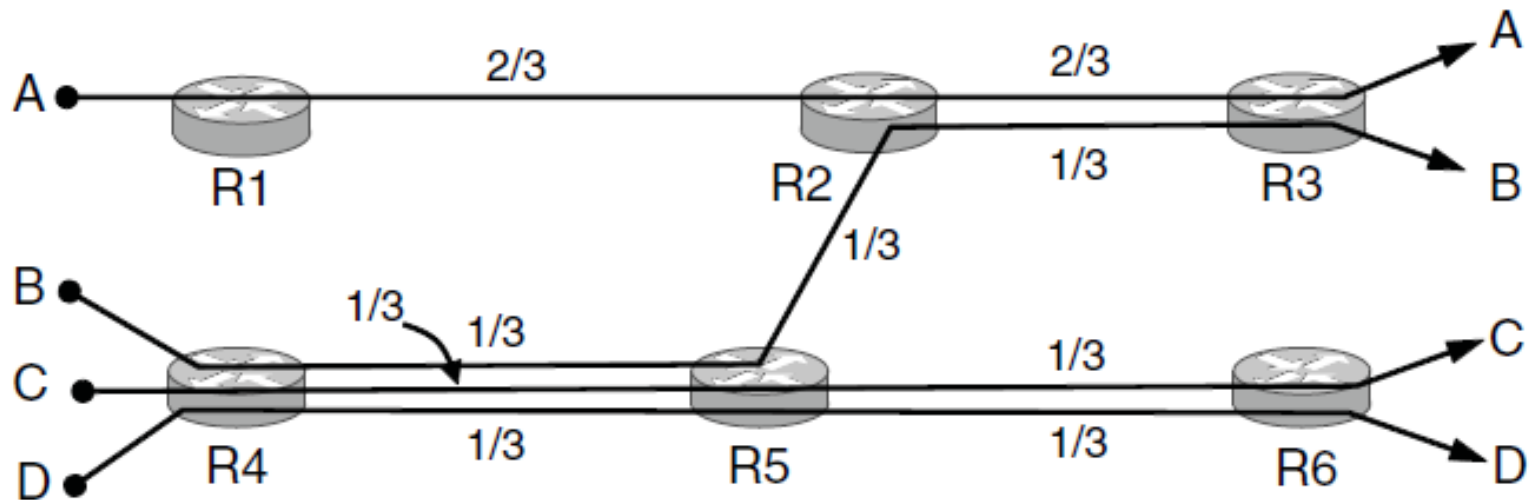
(a)



(b)

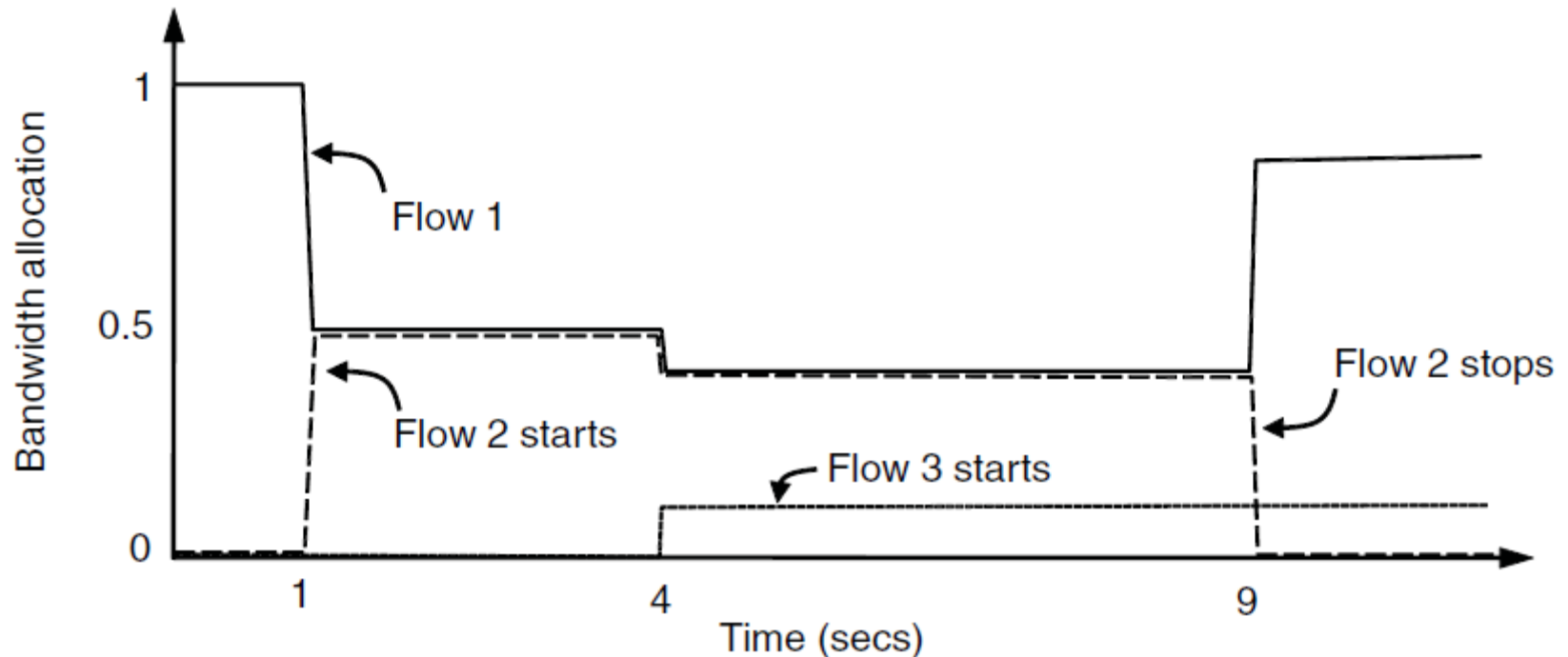
(a) Goodput and (b) delay as a function of offered load
Goodput and delay alone can not serve as indicator of performance. Rather **power = Goodput/delay** is a good indicator.

Desirable Bandwidth Allocation (2)



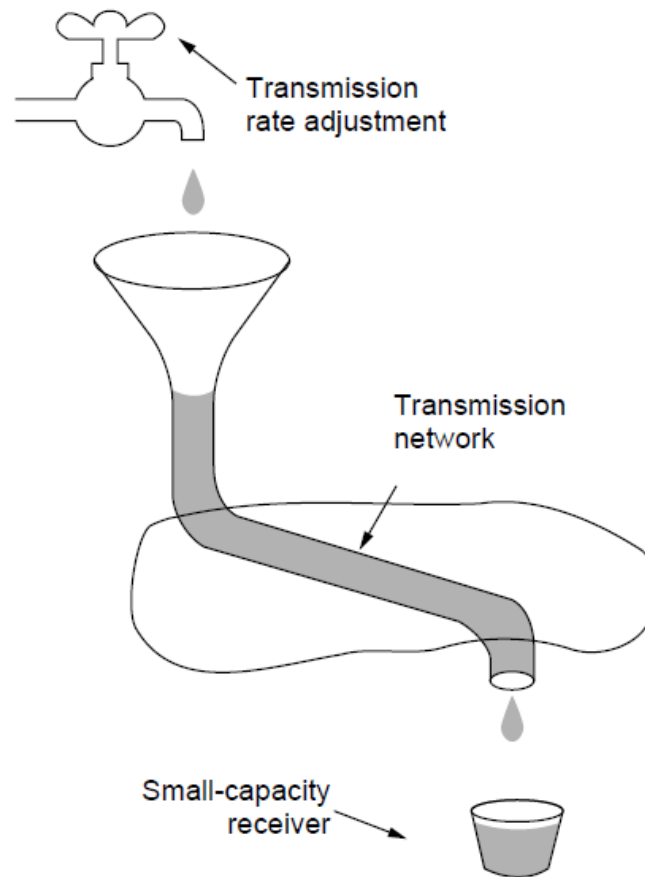
Max-min bandwidth allocation for four flows

Desirable Bandwidth Allocation (3)



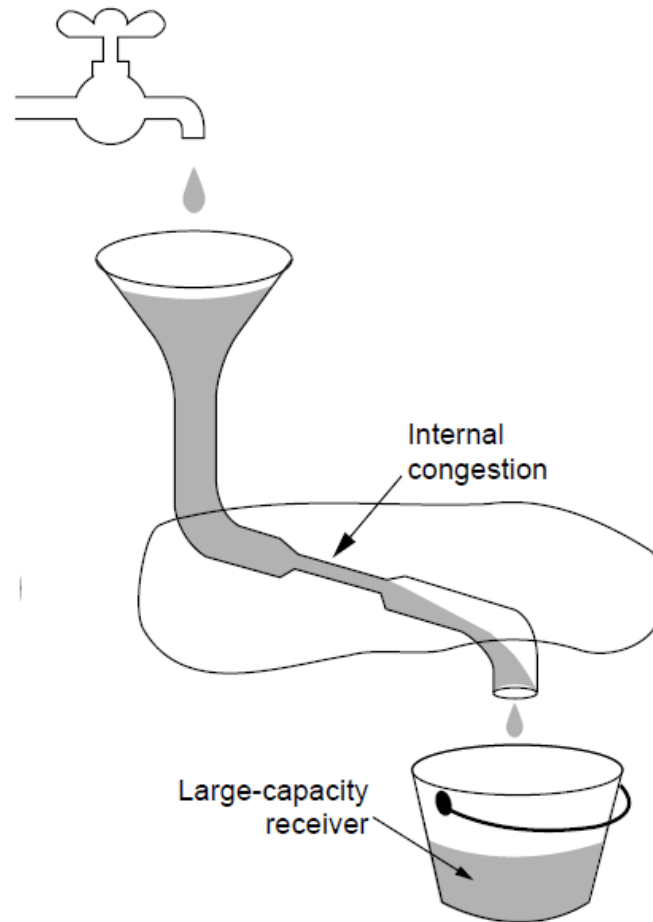
Changing bandwidth allocation over time

Regulating the Sending Rate (1)



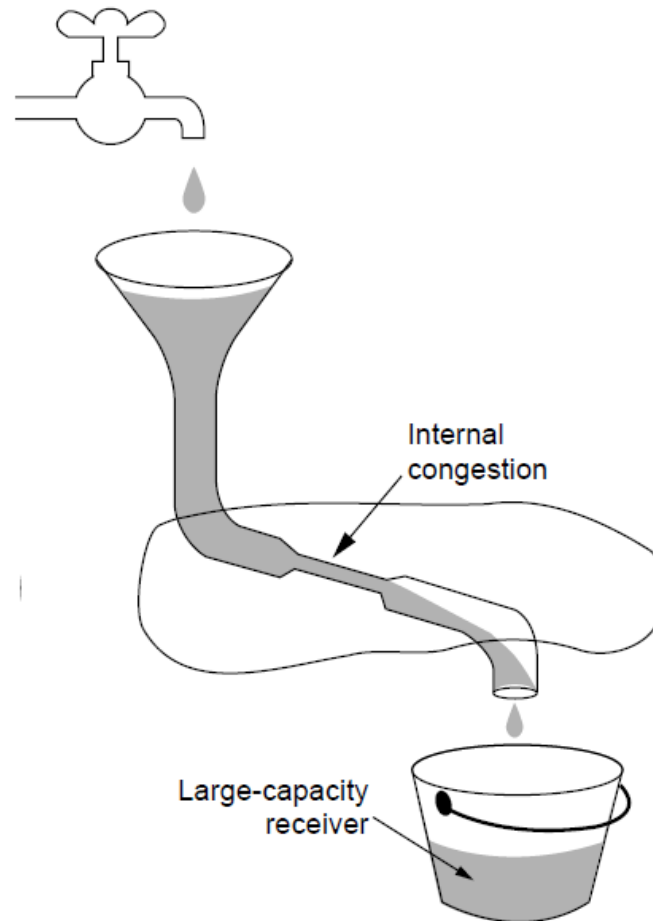
A fast network feeding a low-capacity receiver

Regulating the Sending Rate (2)



A slow network feeding a high-capacity receiver

Regulating the Sending Rate (2)



A slow network feeding a high-capacity receiver

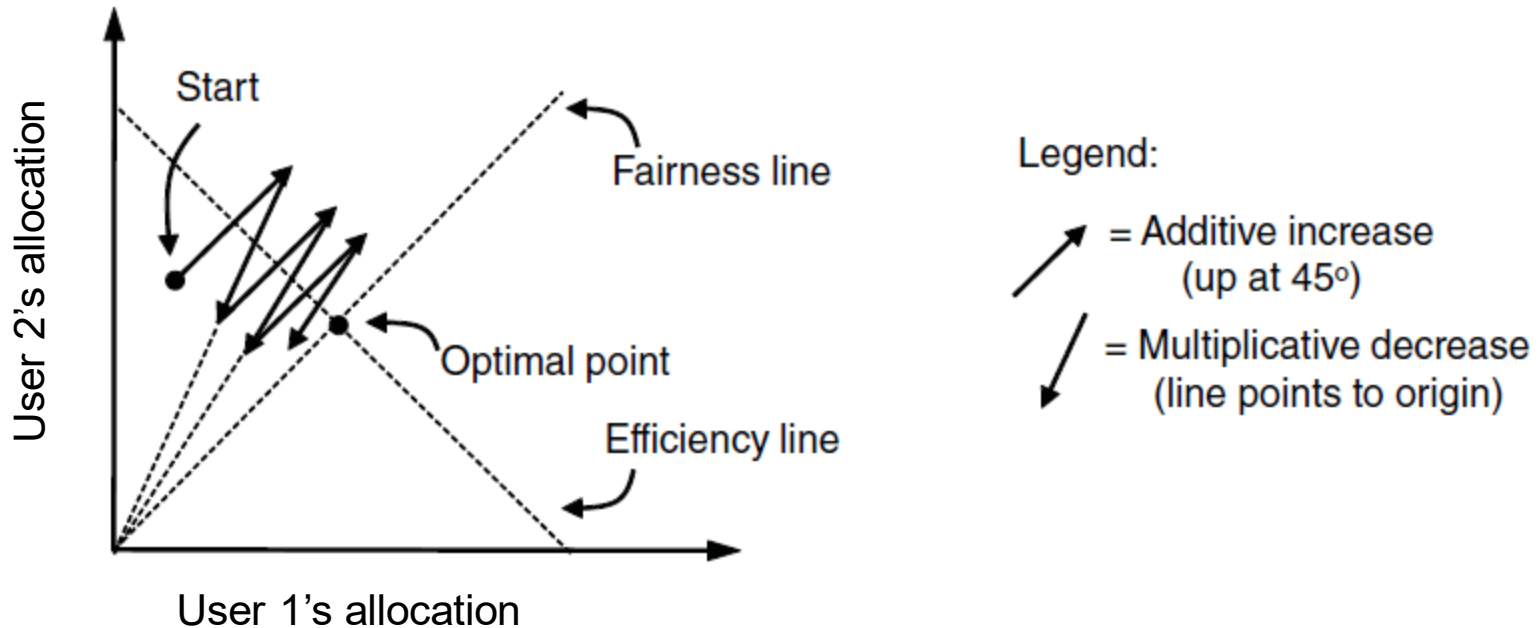
$$S = \min(N, R)$$

Regulating the Sending Rate (3)

Protocol	Signal	Explicit?	Precise?
XCP	Rate to use	Yes	Yes
TCP with ECN	Congestion warning	Yes	No
FAST TCP	End-to-end delay	No	Yes
CUBIC TCP	Packet loss	No	No
TCP	Packet loss	No	No

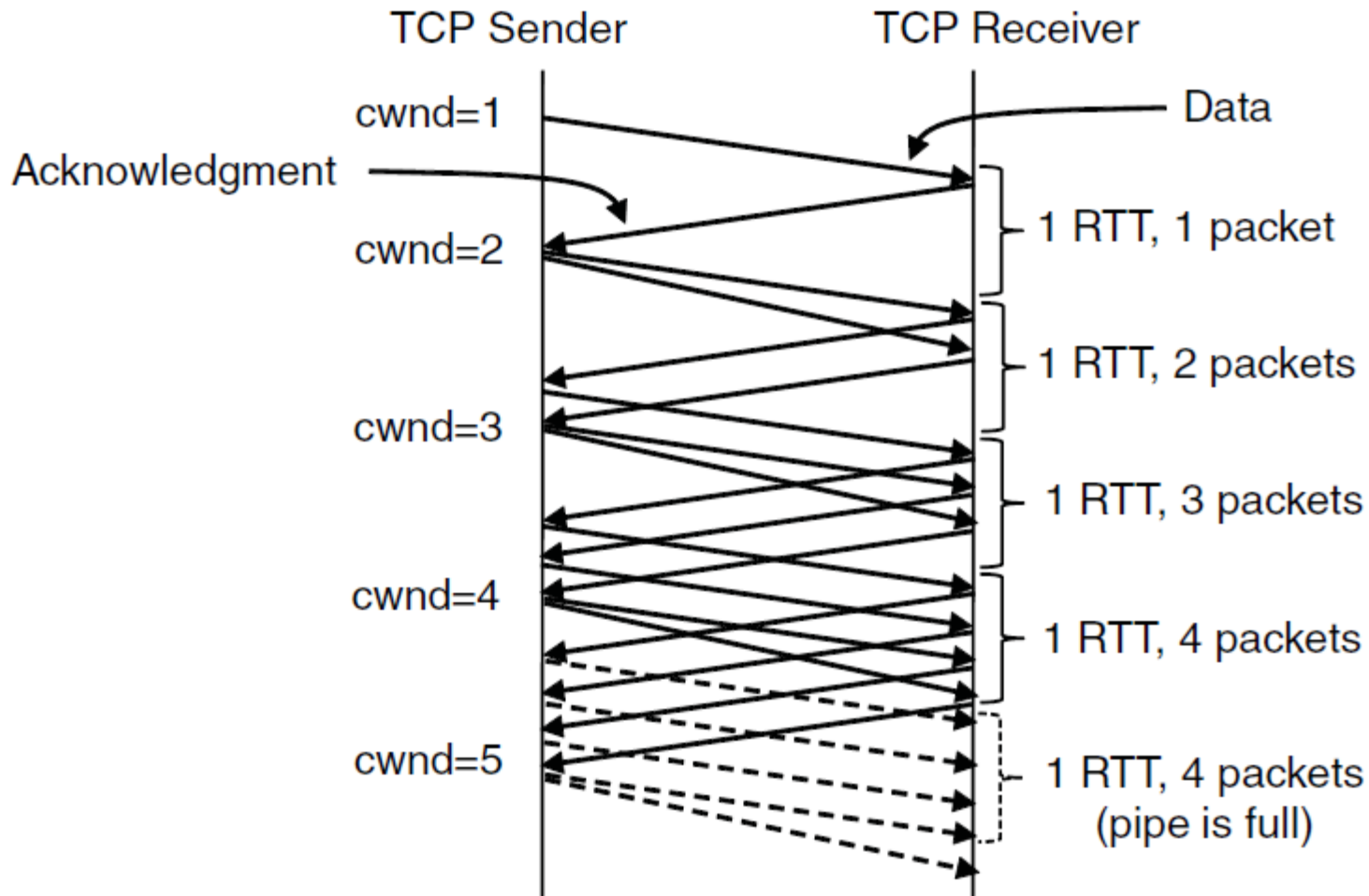
Some congestion control protocols

Regulating the Sending Rate (4)



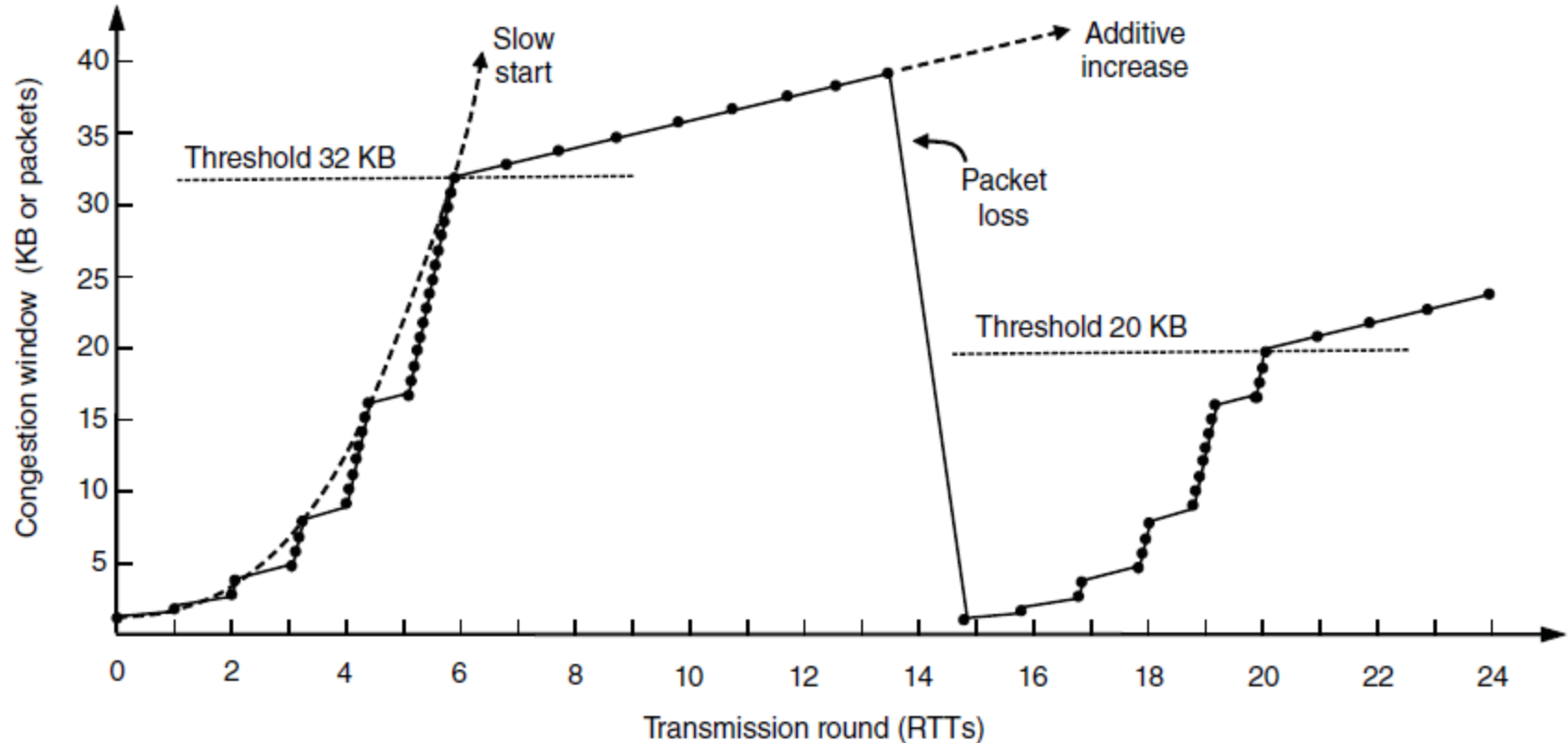
Additive Increase Multiplicative Decrease (AIMD) control law.

TCP Congestion Control (2)



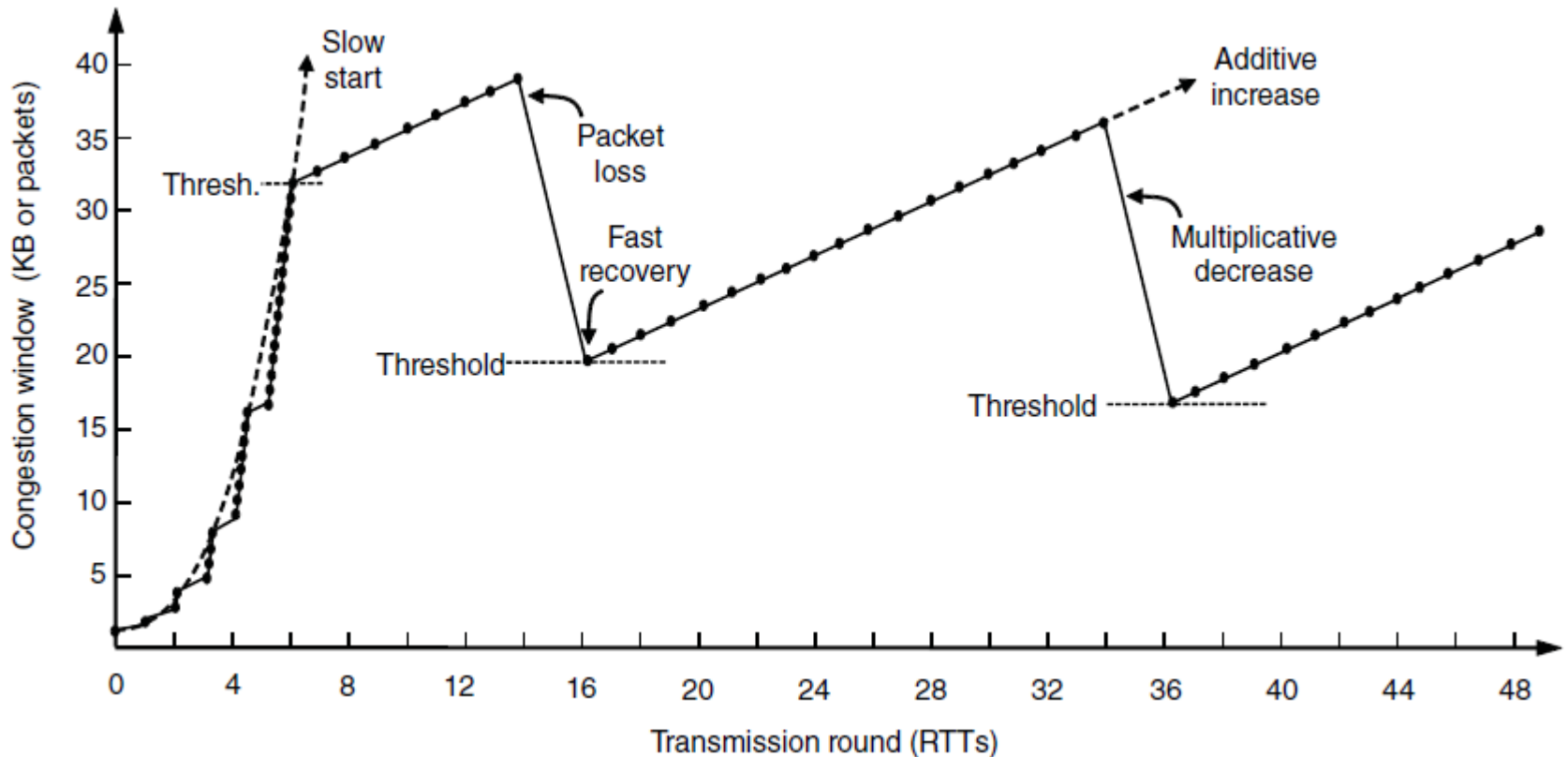
Additive increase from an initial congestion window of 1 segment.

TCP Congestion Control (3)



Slow start followed by additive increase in TCP Tahoe.

TCP Congestion Control (4)



Fast recovery and the sawtooth pattern of TCP Reno.

End

Chapter 6